COMS BC3997 (Applied Cryptography) December 4, 2026

Homework 3: Padding Oracle

Instructor: Eysa Lee *Due: November 9, 2026*

In this assignment, you will explore the security of symmetric key encryption in practice by implementing a Padding Oracle Attack. You will implement a server, which will encrypt messages in CBC mode, and an attacker, which will attempt to decrypt a message byte-by-byte by observing if the server outputs padding error responses.

1 AI, External Resources, and Collaboration

AI Usage. You are permitted the use of AI to assist in syntax and debugging your program assignments. However, you are *not* allowed to have AI code up the entire assignment for you. You are also not allowed to copy code from libraries, implementations, or elsewhere. You should, using your knowledge from class and course materials, come up with the design of your program without the assistance of AI or other external resources. Violation of this policy is considered an honor code violation and may result in a failing grade for the assignment (and possibly the entire course).

Collaborations. Collaborating with classmates is encouraged and allowed. Code walks are individual, and each student will be expected to fully understand and be prepared to talk about their submissions on their own. Collaborations should be acknowledged in the design document submitted alongside the code.

2 Assignment Specification

In this assignment, you will implement a the padding oracle attack on CBC-mode encryption with PKCS#7 padding (see Section 10.3 of the specification). You will implement a **server** and an **adversarial client**. The server will encrypt messages honestly and attempt to decrypt ciphertexts. When decrypting, the server will output a string indicating if the message padding was correct or incorrect. The adversarial client will make use of this padding response to attempt to learn the decryption of a ciphertext.

For this assignment, **you do not need to implement secure channels between the adversary and server**. You may simply send messages between the two. Note that in this system we are *only* dealing with encryption and are *not* authenticating messages. You should *not* be implementing MAC-then-encrypt.

It may be helpful to consult slides 31-35 from the CCA lecture of last semester's Introduction to Cryptography. You are welcome to consult any of the other slides from this course and any of the linked resources.

2.1 Cryptographic Primitives

You will be implementing AES in CBC-mode with padding according PKCS#7.

Recall that to encrypt, messages are split into 16-byte blocks (128-bits). According to the specification we are following, if the byte-length of the message is not a multiple of 16, then padding is appended onto the end of the message to reach the desired length. If the padding required is $b \in [1, 15]$ bytes, then b copies of `0x0b` are appended onto the end of the message. Note if the byte-length of the message *is* a multiple of 16, then a full block of `0x10` (16 copies of the hex representation of 16) is appended onto the end of the message.

During decryption, whoever is decrypting should verify that padding follows the specification. If the padding is *not* valid, the decryptor should output `PADDING_INVALID`. To simplify implementation and testing, the decryptor should output `PADDING_VALID` if the padding is valid. **At no point should the decryptor output the decryption or any partial decryption**. The only outputs it should produce are strings indicating if the padding was correct or incorrect.

3 Main Program API Specification

The following detail the commands your program should handle and expected behavior. Messages will be given as strings, and ciphertexts will be in hex. During testing and grading, we will not input any non-standard or weird character codes.

setup

This should start up your server and an adversarial client as separate processes. The server should additionally sample its encryption key. This command does not produce any output.

encrypt [msg]

The server should encrypt the message `msg` using AES in CBC-mode with PKCS#7 padding. The server should print onto standard out the ciphertext in hex.

check_padding [ct]

Upon receiving a ciphertext in hex, the server should decrypt the ciphertext and validate the PKCS#7 padding. If the padding is valid, output onto standard out `PADDING_VALID`. Otherwise, if the padding is invalid, output onto standard out `PADDING_INVALID`.

run_attack [ct]

The adversarial client should attempt to decrypt `ct` by performing a padding oracle attack against the server. The adversarial client may change bits of the ciphertext and may query the server with modified ciphertexts. The server may respond to decryption queries indicating if the padding is valid or invalid. The server may *not* send any further information to the adversarial client. Once the adversarial client has retrieved the plaintext corresponding to a decryption of the original ciphertext, the adversarial client should strip away the padding and convert the decryption into ASCII. Output onto standard out the decrypted message.

3.1 External Libraries

Allowed. You may use existing implementations of AES, byte manipulation, basic group operations (e.g., exponentiation, modular arithmetic, computing inverses), and sampling randomness.

Not Allowed. You are expected to implement CBC-mode (using existing implementations of AES), PKCS#7 padding and padding checks, and the padding oracle attack. You may not use existing implementations of these components.

Suggested Libraries. You are not required to use the following libraries, but they contain building blocks that may be helpful for your assignment.

- C++: CryptoPP orAWS libcrypto
- Rust: Any of the libraries withing Rust Crypto

4 Submission

Your assignment should be submitted through Gradescope as a ZIP. You will submit the following four components:

1. Source code for your implementation (written in either Rust or C++).
2. A Makefile that compiles your implementation upon running `make`.
3. A main program and file titled "`COMMAND`" that contains on one line the command to run your main program.
4. A PDF titled "design" that includes your name, UNI, **any dependencies**, instructions for how to run your implementation by hand, and information about your implementation. You should include major design decisions, an outline of your code, and any other information you think is relevant to the grading of your assignment. If applicable, please also acknowledge any collaborators you worked with and if/how AI was used in your assignment.

Testing and Grading. For testing and grading, you will also submit a "main" program that can be used to interface with your implementation. Your "main" program may be in any language, but the your implementation for this assignment must be in Rust or C++. (It will likely be easiest to have your main program written in the same language as your implementation.)

The main program will read a sequence of newline delineated commands from standard input ending with EOF. When instructed to, it will display output to standard out. We will only use the main program in the way specified in the API specification and with valid inputs. We are not looking to trip you up on API edge cases.

Codewalks. There will be a 15 minute codewalk where you will be expected to explain not only your code but also design decisions.